```
================================================================================
        SWIFT SUBSET COMPILER - COMPLETE EXECUTION & TEST RESULTS
================================================================================
Generated on: Tue Oct 20 06:46:42 AM IST 2026

--------------------------------------------------------------------------------
```
TEST: Showcase: All Language Features (Specs 1 to 9 combined) (tests/showcase_all_features.swift)
```
--------------------------------------------------------------------------------
Source Code:
// Swift Subset Compiler - All-in-One Comprehensive Showcase
// Covers Specifications 1 through 9

// 1. Primitive Types & 2. Type Inference & 3. Declarations (let/var)
let intVal = 42
let doubleVal = 3.14
let boolVal = true
let charVal = 'S'
let strVal = "Swift"
var mutableCount: Int = 10
mutableCount = mutableCount + 5

// 4. Operators: Arithmetic, Relational, Logical, String Concatenation
let sum = intVal + mutableCount
let isOk = (sum > 50) && boolVal
let fullGreeting = strVal + " Compiler"

// 5. Control Flow: if-else, switch-case-default
if isOk {
    print("Check Passed")
} else {
    print("Check Failed")
}

let choice = 1
switch choice {
    case 1:
        print("Selected 1")
    default:
        print("Default Selected")
}

// 6. Loops: while, repeat-while, for-in
var w = 0
while w < 2 {
    w = w + 1
}

var r = 1
repeat {
    r = r * 2
} while r < 4

for i in 1...3 {
    print(i)
}

// 7. Functions: Named parameters, return types, Void
func addNumbers(a: Int, b: Int) -> Int {
    return a + b
}

func sayHi(target: String) -> Void {
    print("Hi, " + target)
```

```
}

let res = addNumbers(a: 20, b: 30)
sayHi(target: "Viva")

// 8. Arrays: 1D fixed size, subscript notation
var arr = [100, 200, 300]
let elem = arr[0]
arr[1] = 999
print(arr[1])

// 9. User-Defined Types: struct (Exact Assignment Spec)
struct Point {
    var x: Int
    var y: Int
}
var p = Point(x: 3, y: 4)
print(p.x)
p.x = 10
print(p.x)

Compiler Execution Output:
================================================================================
           SWIFT SUBSET COMPILER (LEX & YACC / FLEX & BISON)
================================================================================
Compiling file: tests/showcase_all_features.swift


================================== TOKEN STREAM ==================================
Line    Token Type              Lexeme
--------------------------------------------------------------------------------
5       LET                     let
5       IDENTIFIER              intVal
5       ASSIGN                  =
5       INT_LITERAL             42
6       LET                     let
6       IDENTIFIER              doubleVal
6       ASSIGN                  =
6       DOUBLE_LITERAL          3.14
7       LET                     let
7       IDENTIFIER              boolVal
7       ASSIGN                  =
7       BOOL_LITERAL            true
8       LET                     let
8       IDENTIFIER              charVal
8       ASSIGN                  =
8       CHAR_LITERAL            'S'
9       LET                     let
9       IDENTIFIER              strVal
9       ASSIGN                  =
9       STRING_LITERAL          "Swift"
10      VAR                     var
10      IDENTIFIER              mutableCount
10      COLON                   :
10      TYPE_INT                Int
10      ASSIGN                  =
10      INT_LITERAL             10
11      IDENTIFIER              mutableCount
11      ASSIGN                  =
11      IDENTIFIER              mutableCount
11      PLUS                    +
11      INT_LITERAL             5
14      LET                     let
14      IDENTIFIER              sum
```

```
14      ASSIGN                  =
14      IDENTIFIER              intVal
14      PLUS                    +
14      IDENTIFIER              mutableCount
15      LET                     let
15      IDENTIFIER              isOk
15      ASSIGN                  =
15      LPAREN                  (
15      IDENTIFIER              sum
15      GT                      >
15      INT_LITERAL             50
15      RPAREN                  )
15      AND                     &&
15      IDENTIFIER              boolVal
16      LET                     let
16      IDENTIFIER              fullGreeting
16      ASSIGN                  =
16      IDENTIFIER              strVal
16      PLUS                    +
16      STRING_LITERAL          " Compiler"
19      IF                      if
19      IDENTIFIER              isOk
19      LBRACE                  {
20      PRINT                   print
20      LPAREN                  (
20      STRING_LITERAL          "Check Passed"
20      RPAREN                  )
21      RBRACE                  }
21      ELSE                    else
21      LBRACE                  {
22      PRINT                   print
22      LPAREN                  (
22      STRING_LITERAL          "Check Failed"
22      RPAREN                  )
23      RBRACE                  }
25      LET                     let
25      IDENTIFIER              choice
25      ASSIGN                  =
25      INT_LITERAL             1
26      SWITCH                  switch
26      IDENTIFIER              choice
26      LBRACE                  {
27      CASE                    case
27      INT_LITERAL             1
27      COLON                   :
28      PRINT                   print
28      LPAREN                  (
28      STRING_LITERAL          "Selected 1"
28      RPAREN                  )
29      DEFAULT                 default
29      COLON                   :
30      PRINT                   print
30      LPAREN                  (
30      STRING_LITERAL          "Default Selected"
30      RPAREN                  )
31      RBRACE                  }
34      VAR                     var
34      IDENTIFIER              w
34      ASSIGN                  =
34      INT_LITERAL             0
35      WHILE                   while
35      IDENTIFIER              w
35      LT                      <
35      INT_LITERAL             2
```

```
35      LBRACE                  {
36      IDENTIFIER              w
36      ASSIGN                  =
36      IDENTIFIER              w
36      PLUS                    +
36      INT_LITERAL             1
37      RBRACE                  }
39      VAR                     var
39      IDENTIFIER              r
39      ASSIGN                  =
39      INT_LITERAL             1
40      REPEAT                  repeat
40      LBRACE                  {
41      IDENTIFIER              r
41      ASSIGN                  =
41      IDENTIFIER              r
41      STAR                    *
41      INT_LITERAL             2
42      RBRACE                  }
42      WHILE                   while
42      IDENTIFIER              r
42      LT                      <
42      INT_LITERAL             4
44      FOR                     for
44      IDENTIFIER              i
44      IN                      in
44      INT_LITERAL             1
44      DOTDOTDOT               ...
44      INT_LITERAL             3
44      LBRACE                  {
45      PRINT                   print
45      LPAREN                  (
45      IDENTIFIER              i
45      RPAREN                  )
46      RBRACE                  }
49      FUNC                    func
49      IDENTIFIER              addNumbers
49      LPAREN                  (
49      IDENTIFIER              a
49      COLON                   :
49      TYPE_INT                Int
49      COMMA                   ,
49      IDENTIFIER              b
49      COLON                   :
49      TYPE_INT                Int
49      RPAREN                  )
49      ARROW                   ->
49      TYPE_INT                Int
49      LBRACE                  {
50      RETURN                  return
50      IDENTIFIER              a
50      PLUS                    +
50      IDENTIFIER              b
51      RBRACE                  }
53      FUNC                    func
53      IDENTIFIER              sayHi
53      LPAREN                  (
53      IDENTIFIER              target
53      COLON                   :
53      TYPE_STRING             String
53      RPAREN                  )
53      ARROW                   ->
53      TYPE_VOID               Void
53      LBRACE                  {
```

```
54      PRINT                   print
54      LPAREN                  (
54      STRING_LITERAL          "Hi, "
54      PLUS                    +
54      IDENTIFIER              target
54      RPAREN                  )
55      RBRACE                  }
57      LET                     let
57      IDENTIFIER              res
57      ASSIGN                  =
57      IDENTIFIER              addNumbers
57      LPAREN                  (
57      IDENTIFIER              a
57      COLON                   :
57      INT_LITERAL             20
57      COMMA                   ,
57      IDENTIFIER              b
57      COLON                   :
57      INT_LITERAL             30
57      RPAREN                  )
58      IDENTIFIER              sayHi
58      LPAREN                  (
58      IDENTIFIER              target
58      COLON                   :
58      STRING_LITERAL          "Viva"
58      RPAREN                  )
61      VAR                     var
61      IDENTIFIER              arr
61      ASSIGN                  =
61      LBRACKET                [
61      INT_LITERAL             100
61      COMMA                   ,
61      INT_LITERAL             200
61      COMMA                   ,
61      INT_LITERAL             300
61      RBRACKET                ]
62      LET                     let
62      IDENTIFIER              elem
62      ASSIGN                  =
62      IDENTIFIER              arr
62      LBRACKET                [
62      INT_LITERAL             0
62      RBRACKET                ]
63      IDENTIFIER              arr
63      LBRACKET                [
63      INT_LITERAL             1
63      RBRACKET                ]
63      ASSIGN                  =
63      INT_LITERAL             999
64      PRINT                   print
64      LPAREN                  (
64      IDENTIFIER              arr
64      LBRACKET                [
64      INT_LITERAL             1
64      RBRACKET                ]
64      RPAREN                  )
67      STRUCT                  struct
67      IDENTIFIER              Point
67      LBRACE                  {
68      VAR                     var
68      IDENTIFIER              x
68      COLON                   :
68      TYPE_INT                Int
69      VAR                     var
```

```
69      IDENTIFIER              y
69      COLON                   :
69      TYPE_INT                Int
70      RBRACE                  }
71      VAR                     var
71      IDENTIFIER              p
71      ASSIGN                  =
71      IDENTIFIER              Point
71      LPAREN                  (
71      IDENTIFIER              x
71      COLON                   :
71      INT_LITERAL             3
71      COMMA                   ,
71      IDENTIFIER              y
71      COLON                   :
71      INT_LITERAL             4
71      RPAREN                  )
72      PRINT                   print
72      LPAREN                  (
72      IDENTIFIER              p
72      DOT                     .
72      IDENTIFIER              x
72      RPAREN                  )
73      IDENTIFIER              p
73      DOT                     .
73      IDENTIFIER              x
73      ASSIGN                  =
73      INT_LITERAL             10
74      PRINT                   print
74      LPAREN                  (
74      IDENTIFIER              p
74      DOT                     .
74      IDENTIFIER              x
74      RPAREN                  )
================================================================================
```

```
================================ PARSING MESSAGES ================================
  [PARSER] Variable declaration (let with inference): intVal
  [PARSER] Variable declaration (let with inference): doubleVal
  [PARSER] Variable declaration (let with inference): boolVal
  [PARSER] Variable declaration (let with inference): charVal
  [PARSER] Variable declaration (let with inference): strVal
  [PARSER] Variable declaration (var with type annotation): mutableCount: Int
  [PARSER] Assignment to variable: mutableCount
  [PARSER] Variable declaration (let with inference): sum
  [PARSER] Variable declaration (let with inference): isOk
  [PARSER] Variable declaration (let with inference): fullGreeting
  [PARSER] Print statement: print("Check Passed")
  [PARSER] Print statement: print("Check Failed")
  [PARSER] Variable declaration (let with inference): choice
  [PARSER] Switch statement on expression: choice
  [PARSER] Print statement: print("Selected 1")
  [PARSER] Print statement: print("Default Selected")
  [PARSER] Default case in switch
  [PARSER] Variable declaration (var with inference): w
  [PARSER] Assignment to variable: w
  [PARSER] Variable declaration (var with inference): r
  [PARSER] Assignment to variable: r
  [PARSER] Print statement: print(i)
  [PARSER] Return statement with value: t12
  [PARSER] Function declaration: addNumbers -> Int
  [PARSER] Print statement: print(t13)
  [PARSER] Function declaration: sayHi -> Void
```

```
  [PARSER] Variable declaration (let with inference): res
  [PARSER] Variable declaration (var with inference): arr
  [PARSER] Variable declaration (let with inference): elem
  [PARSER] Assignment to array element: arr[]
  [PARSER] Print statement: print(t17)
  [PARSER] Struct declaration: Point
  [PARSER] Variable declaration (var with inference): p
  [PARSER] Print statement: print(t19)
  [PARSER] Assignment to struct member: p.x
  [PARSER] Print statement: print(t20)
  [PARSER] Program parsing completed successfully.
================================================================================


================================== SYMBOL TABLE ==================================
Identifier     Kind        Type              Scope     Line
--------------------------------------------------------------------------------
arr            var (mut)   [Int]             0         62
boolVal        let (const) Bool              0         8
charVal        let (const) Character         0         9
choice         let (const) Int               0         26
doubleVal      let (const) Double            0         7
elem           let (const) Int               0         63
fullGreeting   let (const) String            0         19
intVal         let (const) Int               0         6
isOk           let (const) Bool              0         16
mutableCount   var (mut)   Int               0         11
p              var (mut)   Point             0         72
r              var (mut)   Int               0         40
res            let (const) Int               0         58
strVal         let (const) String            0         10
sum            let (const) Int               0         15
w              var (mut)   Int               0         35

--- Functions ---
func addNumbers(a: Int, b: Int) -> Int [Line 49]
func sayHi(target: String) -> Void [Line 53]

--- Structs ---
struct Point {
    var x: Int
    var y: Int
} [Line 71]
================================================================================


======================== THREE ADDRESS CODE (TAC) ========================
1:        intVal = 42
2:        doubleVal = 3.140000
3:        boolVal = true
4:        charVal = 'S'
5:        strVal = "Swift"
6:        mutableCount = 10
7:        t0 = mutableCount + 5
8:        mutableCount = t0
9:        t1 = intVal + mutableCount
10:       sum = t1
11:       t2 = sum > 50
12:       t3 = t2 && boolVal
13:       isOk = t3
14:       t4 = strVal + " Compiler"
15:       fullGreeting = t4
16:       ifFalse isOk goto L0
17:       print "Check Passed"
```

```
18:       goto L1
19:   L0:
20:       print "Check Failed"
21:   L1:
22:       choice = 1
23:       t5 = choice == 1
24:       ifFalse t5 goto L3
25:       print "Selected 1"
26:       goto L2
27:   L3:
28:       print "Default Selected"
29:   L2:
30:       w = 0
31:   L4:
32:       t6 = w < 2
33:       ifFalse t6 goto L5
34:       t7 = w + 1
35:       w = t7
36:       goto L4
37:   L5:
38:       r = 1
39:   L6:
40:       t8 = r * 2
41:       r = t8
42:       t9 = r < 4
43:       if t9 goto L6
44:       i = 1
45:   L7:
46:       t10 = i <= 3
47:       ifFalse t10 goto L8
48:       print i
49:       t11 = i + 1
50:       i = t11
51:       goto L7
52:   L8:
53:   func_addNumbers:
54:       t12 = a + b
55:       return t12
56:       return
57:   func_sayHi:
58:       t13 = "Hi, " + target
59:       print t13
60:       return
61:       param 20
62:       param 30
63:       t14 = call func_addNumbers, 2
64:       res = t14
65:       param "Viva"
66:       call func_sayHi, 1
67:       t15[0] = 100
68:       t15[1] = 200
69:       t15[2] = 300
70:       arr = t15
71:       t16 = arr[0]
72:       elem = t16
73:       arr[1] = 999
74:       t17 = arr[1]
75:       print t17
76:       t18.x = 3
77:       t18.y = 4
78:       p = t18
79:       t19 = p.x
80:       print t19
81:       p.x = 10
```

```
82:       t20 = p.x
83:       print t20
================================================================================


>>> Compilation finished successfully with 0 errors.
STATUS: Compilation SUCCEEDED without errors.

--------------------------------------------------------------------------------
TEST: Test 1: Primitives, Type Inference, Operators & String Concatenation (tests/test1_pr
    imitive_and_inference.swift)
--------------------------------------------------------------------------------
Source Code:
// Test 1: Primitive Types, Type Inference, Operators & String Concatenation

// Type Inference
let anInt = 42
let aDouble = 3.14159
let aBool = true
let aChar = 'S'
let aString = "Hello Swift"

// Explicit Type Annotations
var counter: Int = 10
var ratio: Double = 0.75
var active: Bool = false
var letter: Character = 'Z'
var greeting: String = "Hello"

// Arithmetic Operators
var sum = anInt + counter
var diff = anInt - counter
var prod = counter * 5
var quot = anInt / 2
var rem = anInt % 10

// String Concatenation using '+'
let part1 = "Hello, "
let part2 = "World!"
let message = part1 + part2

// Relational and Logical Operators
let isGreater = sum > diff
let isEqual = counter == 10
let complexCondition = (isGreater && isEqual) || !active

print(sum)
print(message)
print(complexCondition)

Compiler Execution Output:
================================================================================
           SWIFT SUBSET COMPILER (LEX & YACC / FLEX & BISON)
================================================================================
Compiling file: tests/test1_primitive_and_inference.swift


================================== TOKEN STREAM ==================================
Line    Token Type              Lexeme
--------------------------------------------------------------------------------
4       LET                     let
4       IDENTIFIER              anInt
4       ASSIGN                  =
4       INT_LITERAL             42
```

```
5       LET                     let
5       IDENTIFIER              aDouble
5       ASSIGN                  =
5       DOUBLE_LITERAL          3.14159
6       LET                     let
6       IDENTIFIER              aBool
6       ASSIGN                  =
6       BOOL_LITERAL            true
7       LET                     let
7       IDENTIFIER              aChar
7       ASSIGN                  =
7       CHAR_LITERAL            'S'
8       LET                     let
8       IDENTIFIER              aString
8       ASSIGN                  =
8       STRING_LITERAL          "Hello Swift"
11      VAR                     var
11      IDENTIFIER              counter
11      COLON                   :
11      TYPE_INT                Int
11      ASSIGN                  =
11      INT_LITERAL             10
12      VAR                     var
12      IDENTIFIER              ratio
12      COLON                   :
12      TYPE_DOUBLE             Double
12      ASSIGN                  =
12      DOUBLE_LITERAL          0.75
13      VAR                     var
13      IDENTIFIER              active
13      COLON                   :
13      TYPE_BOOL               Bool
13      ASSIGN                  =
13      BOOL_LITERAL            false
14      VAR                     var
14      IDENTIFIER              letter
14      COLON                   :
14      TYPE_CHAR               Character
14      ASSIGN                  =
14      CHAR_LITERAL            'Z'
15      VAR                     var
15      IDENTIFIER              greeting
15      COLON                   :
15      TYPE_STRING             String
15      ASSIGN                  =
15      STRING_LITERAL          "Hello"
18      VAR                     var
18      IDENTIFIER              sum
18      ASSIGN                  =
18      IDENTIFIER              anInt
18      PLUS                    +
18      IDENTIFIER              counter
19      VAR                     var
19      IDENTIFIER              diff
19      ASSIGN                  =
19      IDENTIFIER              anInt
19      MINUS                   -
19      IDENTIFIER              counter
20      VAR                     var
20      IDENTIFIER              prod
20      ASSIGN                  =
20      IDENTIFIER              counter
20      STAR                    *
20      INT_LITERAL             5
```

```
21      VAR                     var
21      IDENTIFIER              quot
21      ASSIGN                  =
21      IDENTIFIER              anInt
21      SLASH                   /
21      INT_LITERAL             2
22      VAR                     var
22      IDENTIFIER              rem
22      ASSIGN                  =
22      IDENTIFIER              anInt
22      PERCENT                 %
22      INT_LITERAL             10
25      LET                     let
25      IDENTIFIER              part1
25      ASSIGN                  =
25      STRING_LITERAL          "Hello, "
26      LET                     let
26      IDENTIFIER              part2
26      ASSIGN                  =
26      STRING_LITERAL          "World!"
27      LET                     let
27      IDENTIFIER              message
27      ASSIGN                  =
27      IDENTIFIER              part1
27      PLUS                    +
27      IDENTIFIER              part2
30      LET                     let
30      IDENTIFIER              isGreater
30      ASSIGN                  =
30      IDENTIFIER              sum
30      GT                      >
30      IDENTIFIER              diff
31      LET                     let
31      IDENTIFIER              isEqual
31      ASSIGN                  =
31      IDENTIFIER              counter
31      EQ                      ==
31      INT_LITERAL             10
32      LET                     let
32      IDENTIFIER              complexCondition
32      ASSIGN                  =
32      LPAREN                  (
32      IDENTIFIER              isGreater
32      AND                     &&
32      IDENTIFIER              isEqual
32      RPAREN                  )
32      OR                      ||
32      NOT                     !
32      IDENTIFIER              active
34      PRINT                   print
34      LPAREN                  (
34      IDENTIFIER              sum
34      RPAREN                  )
35      PRINT                   print
35      LPAREN                  (
35      IDENTIFIER              message
35      RPAREN                  )
36      PRINT                   print
36      LPAREN                  (
36      IDENTIFIER              complexCondition
36      RPAREN                  )
================================================================================
```

```
================================ PARSING MESSAGES ================================
  [PARSER] Variable declaration (let with inference): anInt
  [PARSER] Variable declaration (let with inference): aDouble
  [PARSER] Variable declaration (let with inference): aBool
  [PARSER] Variable declaration (let with inference): aChar
  [PARSER] Variable declaration (let with inference): aString
  [PARSER] Variable declaration (var with type annotation): counter: Int
  [PARSER] Variable declaration (var with type annotation): ratio: Double
  [PARSER] Variable declaration (var with type annotation): active: Bool
  [PARSER] Variable declaration (var with type annotation): letter: Character
  [PARSER] Variable declaration (var with type annotation): greeting: String
  [PARSER] Variable declaration (var with inference): sum
  [PARSER] Variable declaration (var with inference): diff
  [PARSER] Variable declaration (var with inference): prod
  [PARSER] Variable declaration (var with inference): quot
  [PARSER] Variable declaration (var with inference): rem
  [PARSER] Variable declaration (let with inference): part1
  [PARSER] Variable declaration (let with inference): part2
  [PARSER] Variable declaration (let with inference): message
  [PARSER] Variable declaration (let with inference): isGreater
  [PARSER] Variable declaration (let with inference): isEqual
  [PARSER] Variable declaration (let with inference): complexCondition
  [PARSER] Print statement: print(sum)
  [PARSER] Print statement: print(message)
  [PARSER] Print statement: print(complexCondition)
  [PARSER] Program parsing completed successfully.
================================================================================


================================ SYMBOL TABLE ================================
Identifier     Kind        Type              Scope     Line
--------------------------------------------------------------------------
aBool          let (const) Bool              0         7
aChar          let (const) Character         0         8
aDouble        let (const) Double            0         6
aString        let (const) String            0         11
active         var (mut)   Bool              0         14
anInt          let (const) Int               0         5
complexConditionlet (const) Bool              0         34
counter        var (mut)   Int               0         12
diff           var (mut)   Int               0         20
greeting       var (mut)   String            0         18
isEqual        let (const) Bool              0         32
isGreater      let (const) Bool              0         31
letter         var (mut)   Character         0         15
message        let (const) String            0         30
part1          let (const) String            0         26
part2          let (const) String            0         27
prod           var (mut)   Int               0         21
quot           var (mut)   Int               0         22
ratio          var (mut)   Double            0         13
rem            var (mut)   Int               0         25
sum            var (mut)   Int               0         19
================================================================================


======================== THREE ADDRESS CODE (TAC) ========================
1:        anInt = 42
2:        aDouble = 3.141590
3:        aBool = true
4:        aChar = 'S'
5:        aString = "Hello Swift"
6:        counter = 10
7:        ratio = 0.750000
```

```
8:        active = false
9:        letter = ’Z’
10:       greeting = "Hello"
11:       t0 = anInt + counter
12:       sum = t0
13:       t1 = anInt - counter
14:       diff = t1
15:       t2 = counter * 5
16:       prod = t2
17:       t3 = anInt / 2
18:       quot = t3
19:       t4 = anInt % 10
20:       rem = t4
21:       part1 = "Hello, "
22:       part2 = "World!"
23:       t5 = part1 + part2
24:       message = t5
25:       t6 = sum > diff
26:       isGreater = t6
27:       t7 = counter == 10
28:       isEqual = t7
29:       t8 = isGreater && isEqual
30:       t9 = !active
31:       t10 = t8 || t9
32:       complexCondition = t10
33:       print sum
34:       print message
35:       print complexCondition
================================================================================


>>> Compilation finished successfully with 0 errors.
STATUS: Compilation SUCCEEDED without errors.

--------------------------------------------------------------------------------
TEST: Test 2: Variable Declarations (let immutable vs var mutable) (tests/test2_immutabili
    ty_let_var.swift)
--------------------------------------------------------------------------------
Source Code:
// Test 2: Variable Declarations (let immutable vs var mutable)

// Immutable declarations
let maxScore = 100
let appName: String = "SwiftCompiler"

// Mutable declarations
var currentScore = 0
var status = "Playing"

// Valid reassignments to mutable variables
currentScore = 10
currentScore = currentScore + 15
status = "Completed"

print(appName)
print(currentScore)
print(status)

Compiler Execution Output:
================================================================================
           SWIFT SUBSET COMPILER (LEX & YACC / FLEX & BISON)
================================================================================
Compiling file: tests/test2_immutability_let_var.swift
```

```
================================ TOKEN STREAM ================================
Line    Token Type              Lexeme
--------------------------------------------------------------------------------
4       LET                     let
4       IDENTIFIER              maxScore
4       ASSIGN                  =
4       INT_LITERAL             100
5       LET                     let
5       IDENTIFIER              appName
5       COLON                   :
5       TYPE_STRING             String
5       ASSIGN                  =
5       STRING_LITERAL          "SwiftCompiler"
8       VAR                     var
8       IDENTIFIER              currentScore
8       ASSIGN                  =
8       INT_LITERAL             0
9       VAR                     var
9       IDENTIFIER              status
9       ASSIGN                  =
9       STRING_LITERAL          "Playing"
12      IDENTIFIER              currentScore
12      ASSIGN                  =
12      INT_LITERAL             10
13      IDENTIFIER              currentScore
13      ASSIGN                  =
13      IDENTIFIER              currentScore
13      PLUS                    +
13      INT_LITERAL             15
14      IDENTIFIER              status
14      ASSIGN                  =
14      STRING_LITERAL          "Completed"
16      PRINT                   print
16      LPAREN                  (
16      IDENTIFIER              appName
16      RPAREN                  )
17      PRINT                   print
17      LPAREN                  (
17      IDENTIFIER              currentScore
17      RPAREN                  )
18      PRINT                   print
18      LPAREN                  (
18      IDENTIFIER              status
18      RPAREN                  )
================================================================================


============================== PARSING MESSAGES ==============================
  [PARSER] Variable declaration (let with inference): maxScore
  [PARSER] Variable declaration (let with type annotation): appName: String
  [PARSER] Variable declaration (var with inference): currentScore
  [PARSER] Variable declaration (var with inference): status
  [PARSER] Assignment to variable: currentScore
  [PARSER] Assignment to variable: currentScore
  [PARSER] Assignment to variable: status
  [PARSER] Print statement: print(appName)
  [PARSER] Print statement: print(currentScore)
  [PARSER] Print statement: print(status)
  [PARSER] Program parsing completed successfully.
================================================================================


============================== SYMBOL TABLE ==============================
```

```
Identifier     Kind        Type              Scope     Line
--------------------------------------------------------------------------------
appName        let (const) String            0         8
currentScore   var (mut)   Int               0         9
maxScore       let (const) Int               0         5
status         var (mut)   String            0         12
================================================================================


======================== THREE ADDRESS CODE (TAC) ========================
1:        maxScore = 100
2:        appName = "SwiftCompiler"
3:        currentScore = 0
4:        status = "Playing"
5:        currentScore = 10
6:        t0 = currentScore + 15
7:        currentScore = t0
8:        status = "Completed"
9:        print appName
10:       print currentScore
11:       print status
================================================================================


>>> Compilation finished successfully with 0 errors.
STATUS: Compilation SUCCEEDED without errors.

--------------------------------------------------------------------------------
TEST: Test 3: Control Flow (if, else, switch, case, default) (tests/test3_control_flow.swi
    ft)
--------------------------------------------------------------------------------
Source Code:
// Test 3: Control Flow (if, else, switch with case, default)

let score = 85

// if-else construct
if score >= 90 {
    print("Grade: A")
} else {
    if score >= 80 {
        print("Grade: B")
    } else {
        print("Grade: C or below")
    }
}

// switch construct with case and default
let code = 2
switch code {
    case 1:
        print("Option 1 selected")
    case 2:
        print("Option 2 selected")
    case 3:
        print("Option 3 selected")
    default:
        print("Default fallback option")
}

Compiler Execution Output:
================================================================================
           SWIFT SUBSET COMPILER (LEX & YACC / FLEX & BISON)
================================================================================
```

```
Compiling file: tests/test3_control_flow.swift


================================== TOKEN STREAM ==================================
Line    Token Type              Lexeme
--------------------------------------------------------------------------------
3       LET                     let
3       IDENTIFIER              score
3       ASSIGN                  =
3       INT_LITERAL             85
6       IF                      if
6       IDENTIFIER              score
6       GE                      >=
6       INT_LITERAL             90
6       LBRACE                  {
7       PRINT                   print
7       LPAREN                  (
7       STRING_LITERAL          "Grade: A"
7       RPAREN                  )
8       RBRACE                  }
8       ELSE                    else
8       LBRACE                  {
9       IF                      if
9       IDENTIFIER              score
9       GE                      >=
9       INT_LITERAL             80
9       LBRACE                  {
10      PRINT                   print
10      LPAREN                  (
10      STRING_LITERAL          "Grade: B"
10      RPAREN                  )
11      RBRACE                  }
11      ELSE                    else
11      LBRACE                  {
12      PRINT                   print
12      LPAREN                  (
12      STRING_LITERAL          "Grade: C or below"
12      RPAREN                  )
13      RBRACE                  }
14      RBRACE                  }
17      LET                     let
17      IDENTIFIER              code
17      ASSIGN                  =
17      INT_LITERAL             2
18      SWITCH                  switch
18      IDENTIFIER              code
18      LBRACE                  {
19      CASE                    case
19      INT_LITERAL             1
19      COLON                   :
20      PRINT                   print
20      LPAREN                  (
20      STRING_LITERAL          "Option 1 selected"
20      RPAREN                  )
21      CASE                    case
21      INT_LITERAL             2
21      COLON                   :
22      PRINT                   print
22      LPAREN                  (
22      STRING_LITERAL          "Option 2 selected"
22      RPAREN                  )
23      CASE                    case
23      INT_LITERAL             3
23      COLON                   :
```

```
24      PRINT                   print
24      LPAREN                  (
24      STRING_LITERAL          "Option 3 selected"
24      RPAREN                  )
25      DEFAULT                 default
25      COLON                   :
26      PRINT                   print
26      LPAREN                  (
26      STRING_LITERAL          "Default fallback option"
26      RPAREN                  )
27      RBRACE                  }
================================================================================


================================ PARSING MESSAGES ================================
  [PARSER] Variable declaration (let with inference): score
  [PARSER] Print statement: print("Grade: A")
  [PARSER] Print statement: print("Grade: B")
  [PARSER] Print statement: print("Grade: C or below")
  [PARSER] Variable declaration (let with inference): code
  [PARSER] Switch statement on expression: code
  [PARSER] Print statement: print("Option 1 selected")
  [PARSER] Print statement: print("Option 2 selected")
  [PARSER] Print statement: print("Option 3 selected")
  [PARSER] Print statement: print("Default fallback option")
  [PARSER] Default case in switch
  [PARSER] Program parsing completed successfully.
================================================================================


================================ SYMBOL TABLE ================================
Identifier     Kind        Type              Scope     Line
--------------------------------------------------------------------------
code           let (const) Int               0         18
score          let (const) Int               0         6
================================================================================


======================== THREE ADDRESS CODE (TAC) ========================
1:        score = 85
2:        t0 = score >= 90
3:        ifFalse t0 goto L0
4:        print "Grade: A"
5:        goto L1
6:    L0:
7:        t1 = score >= 80
8:        ifFalse t1 goto L2
9:        print "Grade: B"
10:       goto L3
11:   L2:
12:       print "Grade: C or below"
13:   L3:
14:   L1:
15:       code = 2
16:       t2 = code == 1
17:       ifFalse t2 goto L5
18:       print "Option 1 selected"
19:       goto L4
20:   L5:
21:       t3 = code == 2
22:       ifFalse t3 goto L6
23:       print "Option 2 selected"
24:       goto L4
25:   L6:
```

```
26:       t4 = code == 3
27:       ifFalse t4 goto L7
28:       print "Option 3 selected"
29:       goto L4
30:   L7:
31:       print "Default fallback option"
32:   L4:
================================================================================


>>> Compilation finished successfully with 0 errors.
STATUS: Compilation SUCCEEDED without errors.

--------------------------------------------------------------------------------
TEST: Test 4: Loops (while, repeat-while, for-in) (tests/test4_loops.swift)
--------------------------------------------------------------------------------
Source Code:
// Test 4: Loops (while, repeat-while, for-in)

// while loop
var count = 0
while count < 3 {
    print(count)
    count = count + 1
}

// repeat-while loop
var n = 1
repeat {
    print(n)
    n = n * 2
} while n < 8

// for-in loop with range
for i in 1...5 {
    print(i)
}

Compiler Execution Output:
================================================================================
           SWIFT SUBSET COMPILER (LEX & YACC / FLEX & BISON)
================================================================================
Compiling file: tests/test4_loops.swift


================================== TOKEN STREAM ==================================
Line    Token Type              Lexeme
--------------------------------------------------------------------------------
4       VAR                     var
4       IDENTIFIER              count
4       ASSIGN                  =
4       INT_LITERAL             0
5       WHILE                   while
5       IDENTIFIER              count
5       LT                      <
5       INT_LITERAL             3
5       LBRACE                  {
6       PRINT                   print
6       LPAREN                  (
6       IDENTIFIER              count
6       RPAREN                  )
7       IDENTIFIER              count
7       ASSIGN                  =
7       IDENTIFIER              count
```

```
7       PLUS                    +
7       INT_LITERAL             1
8       RBRACE                  }
11      VAR                     var
11      IDENTIFIER              n
11      ASSIGN                  =
11      INT_LITERAL             1
12      REPEAT                  repeat
12      LBRACE                  {
13      PRINT                   print
13      LPAREN                  (
13      IDENTIFIER              n
13      RPAREN                  )
14      IDENTIFIER              n
14      ASSIGN                  =
14      IDENTIFIER              n
14      STAR                    *
14      INT_LITERAL             2
15      RBRACE                  }
15      WHILE                   while
15      IDENTIFIER              n
15      LT                      <
15      INT_LITERAL             8
18      FOR                     for
18      IDENTIFIER              i
18      IN                      in
18      INT_LITERAL             1
18      DOTDOTDOT               ...
18      INT_LITERAL             5
18      LBRACE                  {
19      PRINT                   print
19      LPAREN                  (
19      IDENTIFIER              i
19      RPAREN                  )
20      RBRACE                  }
================================================================================


================================ PARSING MESSAGES ================================
  [PARSER] Variable declaration (var with inference): count
  [PARSER] Print statement: print(count)
  [PARSER] Assignment to variable: count
  [PARSER] Variable declaration (var with inference): n
  [PARSER] Print statement: print(n)
  [PARSER] Assignment to variable: n
  [PARSER] Print statement: print(i)
  [PARSER] Program parsing completed successfully.
================================================================================


================================ SYMBOL TABLE ================================
Identifier     Kind        Type             Scope     Line
--------------------------------------------------------------------------
count          var (mut)   Int              0         5
n              var (mut)   Int              0         12
================================================================================


======================== THREE ADDRESS CODE (TAC) ========================
1:        count = 0
2:    L0:
3:        t0 = count < 3
4:        ifFalse t0 goto L1
5:        print count
```

```
6:        t1 = count + 1
7:        count = t1
8:        goto L0
9:    L1:
10:       n = 1
11:   L2:
12:       print n
13:       t2 = n * 2
14:       n = t2
15:       t3 = n < 8
16:       if t3 goto L2
17:       i = 1
18:   L3:
19:       t4 = i <= 5
20:       ifFalse t4 goto L4
21:       print i
22:       t5 = i + 1
23:       i = t5
24:       goto L3
25:   L4:
================================================================================


>>> Compilation finished successfully with 0 errors.
STATUS: Compilation SUCCEEDED without errors.

--------------------------------------------------------------------------------
TEST: Test 5: Functions (Parameters, Return Types, Named Params, Void) (tests/test5_functi
    ons.swift)
--------------------------------------------------------------------------------
Source Code:
// Test 5: Functions (return values, parameters, named parameters, Void return type)

func add(a: Int, b: Int) -> Int {
    return a + b
}

func multiply(x: Int, y: Int) -> Int {
    let result = x * y
    return result
}

func greet(name: String) -> Void {
    let greeting = "Welcome, " + name
    print(greeting)
}

func sayHello() {
    print("Hello from Void function!")
}

let sum = add(a: 10, b: 20)
let product = multiply(x: 5, y: 6)
greet(name: "Alice")
sayHello()

print(sum)
print(product)

Compiler Execution Output:
================================================================================
           SWIFT SUBSET COMPILER (LEX & YACC / FLEX & BISON)
================================================================================
Compiling file: tests/test5_functions.swift
```

```
================================= TOKEN STREAM ==================================
Line    Token Type              Lexeme
--------------------------------------------------------------------------------
3       FUNC                    func
3       IDENTIFIER              add
3       LPAREN                  (
3       IDENTIFIER              a
3       COLON                   :
3       TYPE_INT                Int
3       COMMA                   ,
3       IDENTIFIER              b
3       COLON                   :
3       TYPE_INT                Int
3       RPAREN                  )
3       ARROW                   ->
3       TYPE_INT                Int
3       LBRACE                  {
4       RETURN                  return
4       IDENTIFIER              a
4       PLUS                    +
4       IDENTIFIER              b
5       RBRACE                  }
7       FUNC                    func
7       IDENTIFIER              multiply
7       LPAREN                  (
7       IDENTIFIER              x
7       COLON                   :
7       TYPE_INT                Int
7       COMMA                   ,
7       IDENTIFIER              y
7       COLON                   :
7       TYPE_INT                Int
7       RPAREN                  )
7       ARROW                   ->
7       TYPE_INT                Int
7       LBRACE                  {
8       LET                     let
8       IDENTIFIER              result
8       ASSIGN                  =
8       IDENTIFIER              x
8       STAR                    *
8       IDENTIFIER              y
9       RETURN                  return
9       IDENTIFIER              result
10      RBRACE                  }
12      FUNC                    func
12      IDENTIFIER              greet
12      LPAREN                  (
12      IDENTIFIER              name
12      COLON                   :
12      TYPE_STRING             String
12      RPAREN                  )
12      ARROW                   ->
12      TYPE_VOID               Void
12      LBRACE                  {
13      LET                     let
13      IDENTIFIER              greeting
13      ASSIGN                  =
13      STRING_LITERAL          "Welcome, "
13      PLUS                    +
13      IDENTIFIER              name
14      PRINT                   print
```

```
14      LPAREN                  (
14      IDENTIFIER              greeting
14      RPAREN                  )
15      RBRACE                  }
17      FUNC                    func
17      IDENTIFIER              sayHello
17      LPAREN                  (
17      RPAREN                  )
17      LBRACE                  {
18      PRINT                   print
18      LPAREN                  (
18      STRING_LITERAL          "Hello from Void function!"
18      RPAREN                  )
19      RBRACE                  }
21      LET                     let
21      IDENTIFIER              sum
21      ASSIGN                  =
21      IDENTIFIER              add
21      LPAREN                  (
21      IDENTIFIER              a
21      COLON                   :
21      INT_LITERAL             10
21      COMMA                   ,
21      IDENTIFIER              b
21      COLON                   :
21      INT_LITERAL             20
21      RPAREN                  )
22      LET                     let
22      IDENTIFIER              product
22      ASSIGN                  =
22      IDENTIFIER              multiply
22      LPAREN                  (
22      IDENTIFIER              x
22      COLON                   :
22      INT_LITERAL             5
22      COMMA                   ,
22      IDENTIFIER              y
22      COLON                   :
22      INT_LITERAL             6
22      RPAREN                  )
23      IDENTIFIER              greet
23      LPAREN                  (
23      IDENTIFIER              name
23      COLON                   :
23      STRING_LITERAL          "Alice"
23      RPAREN                  )
24      IDENTIFIER              sayHello
24      LPAREN                  (
24      RPAREN                  )
26      PRINT                   print
26      LPAREN                  (
26      IDENTIFIER              sum
26      RPAREN                  )
27      PRINT                   print
27      LPAREN                  (
27      IDENTIFIER              product
27      RPAREN                  )
================================================================================


================================ PARSING MESSAGES ================================
  [PARSER] Return statement with value: t0
  [PARSER] Function declaration: add -> Int
  [PARSER] Variable declaration (let with inference): result
```

```
  [PARSER] Return statement with value: result
  [PARSER] Function declaration: multiply -> Int
  [PARSER] Variable declaration (let with inference): greeting
  [PARSER] Print statement: print(greeting)
  [PARSER] Function declaration: greet -> Void
  [PARSER] Print statement: print("Hello from Void function!")
  [PARSER] Function declaration (Void): sayHello
  [PARSER] Variable declaration (let with inference): sum
  [PARSER] Variable declaration (let with inference): product
  [PARSER] Print statement: print(sum)
  [PARSER] Print statement: print(product)
  [PARSER] Program parsing completed successfully.
================================================================================


================================ SYMBOL TABLE ================================
Identifier     Kind        Type              Scope     Line
--------------------------------------------------------------------------
product        let (const) Int               0         23
sum            let (const) Int               0         22

--- Functions ---
func add(a: Int, b: Int) -> Int [Line 3]
func greet(name: String) -> Void [Line 12]
func multiply(x: Int, y: Int) -> Int [Line 7]
func sayHello() -> Void [Line 17]
================================================================================


======================== THREE ADDRESS CODE (TAC) ========================
1:    func_add:
2:        t0 = a + b
3:        return t0
4:        return
5:    func_multiply:
6:        t1 = x * y
7:        result = t1
8:        return result
9:        return
10:   func_greet:
11:       t2 = "Welcome, " + name
12:       greeting = t2
13:       print greeting
14:       return
15:   func_sayHello:
16:       print "Hello from Void function!"
17:       return
18:       param 10
19:       param 20
20:       t3 = call func_add, 2
21:       sum = t3
22:       param 5
23:       param 6
24:       t4 = call func_multiply, 2
25:       product = t4
26:       param "Alice"
27:       call func_greet, 1
28:       call func_sayHello, 0
29:       print sum
30:       print product
================================================================================


>>> Compilation finished successfully with 0 errors.
```

STATUS: Compilation SUCCEEDED without errors.

TEST: Test 6: Arrays (Fixed 1D arrays, Subscript access, Subscript mutation) (tests/test6_arrays.swift)

Source Code:

```
// Test 6: Arrays (1D arrays of fixed size, element access, subscript assignment)

var numbers = [10, 20, 30, 40, 50]

// Read array elements
let first = numbers[0]
let second = numbers[1]
print(first)
print(second)

// Modify array element
numbers[2] = 99
let third = numbers[2]
print(third)

// Array element with expression index
let idx = 3
let fourth = numbers[idx]
print(fourth)

// Iterating over array using for-in loop
for val in numbers {
    print(val)
}
```

Compiler Execution Output:

```
================================================================================
           SWIFT SUBSET COMPILER (LEX & YACC / FLEX & BISON)
================================================================================
Compiling file: tests/test6_arrays.swift


================================== TOKEN STREAM ==================================
Line    Token Type              Lexeme
--------------------------------------------------------------------------------
3       VAR                     var
3       IDENTIFIER              numbers
3       ASSIGN                  =
3       LBRACKET                [
3       INT_LITERAL             10
3       COMMA                   ,
3       INT_LITERAL             20
3       COMMA                   ,
3       INT_LITERAL             30
3       COMMA                   ,
3       INT_LITERAL             40
3       COMMA                   ,
3       INT_LITERAL             50
3       RBRACKET                ]
6       LET                     let
6       IDENTIFIER              first
6       ASSIGN                  =
6       IDENTIFIER              numbers
6       LBRACKET                [
6       INT_LITERAL             0
6       RBRACKET                ]
7       LET                     let
```

```
7       IDENTIFIER              second
7       ASSIGN                  =
7       IDENTIFIER              numbers
7       LBRACKET                [
7       INT_LITERAL             1
7       RBRACKET                ]
8       PRINT                   print
8       LPAREN                  (
8       IDENTIFIER              first
8       RPAREN                  )
9       PRINT                   print
9       LPAREN                  (
9       IDENTIFIER              second
9       RPAREN                  )
12      IDENTIFIER              numbers
12      LBRACKET                [
12      INT_LITERAL             2
12      RBRACKET                ]
12      ASSIGN                  =
12      INT_LITERAL             99
13      LET                     let
13      IDENTIFIER              third
13      ASSIGN                  =
13      IDENTIFIER              numbers
13      LBRACKET                [
13      INT_LITERAL             2
13      RBRACKET                ]
14      PRINT                   print
14      LPAREN                  (
14      IDENTIFIER              third
14      RPAREN                  )
17      LET                     let
17      IDENTIFIER              idx
17      ASSIGN                  =
17      INT_LITERAL             3
18      LET                     let
18      IDENTIFIER              fourth
18      ASSIGN                  =
18      IDENTIFIER              numbers
18      LBRACKET                [
18      IDENTIFIER              idx
18      RBRACKET                ]
19      PRINT                   print
19      LPAREN                  (
19      IDENTIFIER              fourth
19      RPAREN                  )
22      FOR                     for
22      IDENTIFIER              val
22      IN                      in
22      IDENTIFIER              numbers
22      LBRACE                  {
23      PRINT                   print
23      LPAREN                  (
23      IDENTIFIER              val
23      RPAREN                  )
24      RBRACE                  }
================================================================================


================================ PARSING MESSAGES ================================
  [PARSER] Variable declaration (var with inference): numbers
  [PARSER] Variable declaration (let with inference): first
  [PARSER] Variable declaration (let with inference): second
  [PARSER] Print statement: print(first)
```

```
  [PARSER] Print statement: print(second)
  [PARSER] Assignment to array element: numbers[]
  [PARSER] Variable declaration (let with inference): third
  [PARSER] Print statement: print(third)
  [PARSER] Variable declaration (let with inference): idx
  [PARSER] Variable declaration (let with inference): fourth
  [PARSER] Print statement: print(fourth)
  [PARSER] Print statement: print(val)
  [PARSER] Program parsing completed successfully.
================================================================================


================================ SYMBOL TABLE ================================
Identifier     Kind        Type              Scope     Line
--------------------------------------------------------------------------------
first          let (const) Int               0         7
fourth         let (const) Int               0         19
idx            let (const) Int               0         18
numbers        var (mut)   [Int]             0         6
second         let (const) Int               0         8
third          let (const) Int               0         14
================================================================================


======================== THREE ADDRESS CODE (TAC) ========================
1:        t0[0] = 10
2:        t0[1] = 20
3:        t0[2] = 30
4:        t0[3] = 40
5:        t0[4] = 50
6:        numbers = t0
7:        t1 = numbers[0]
8:        first = t1
9:        t2 = numbers[1]
10:       second = t2
11:       print first
12:       print second
13:       numbers[2] = 99
14:       t3 = numbers[2]
15:       third = t3
16:       print third
17:       idx = 3
18:       t4 = numbers[idx]
19:       fourth = t4
20:       print fourth
21:       t5 = 0
22:   L0:
23:       t6 = t5 < 5
24:       ifFalse t6 goto L1
25:       val = numbers[t5]
26:       print val
27:       t7 = t5 + 1
28:       t5 = t7
29:       goto L0
30:   L1:
================================================================================


>>> Compilation finished successfully with 0 errors.
STATUS: Compilation SUCCEEDED without errors.

--------------------------------------------------------------------------------
```

TEST: Test 7: Structs (Value semantics, Member access with '.', Assignment example) (tests /test7_structs.swift)

```
--------------------------------------------------------------------------------
Source Code:
// Test 7: User-Defined Types (struct)
// Assignment Objective Example Requirement 9

struct Point {
    var x: Int
    var y: Int
}

var p = Point(x: 3, y: 4)
print(p.x)
print(p.y)

// Field mutation on mutable struct
p.x = 10
p.y = 20
print(p.x)
print(p.y)

// Immutable struct
let origin = Point(x: 0, y: 0)
print(origin.x)
print(origin.y)

Compiler Execution Output:
================================================================================
           SWIFT SUBSET COMPILER (LEX & YACC / FLEX & BISON)
================================================================================
Compiling file: tests/test7_structs.swift


================================== TOKEN STREAM ==================================
Line    Token Type              Lexeme
--------------------------------------------------------------------------------
4       STRUCT                  struct
4       IDENTIFIER              Point
4       LBRACE                  {
5       VAR                     var
5       IDENTIFIER              x
5       COLON                   :
5       TYPE_INT                Int
6       VAR                     var
6       IDENTIFIER              y
6       COLON                   :
6       TYPE_INT                Int
7       RBRACE                  }
9       VAR                     var
9       IDENTIFIER              p
9       ASSIGN                  =
9       IDENTIFIER              Point
9       LPAREN                  (
9       IDENTIFIER              x
9       COLON                   :
9       INT_LITERAL             3
9       COMMA                   ,
9       IDENTIFIER              y
9       COLON                   :
9       INT_LITERAL             4
9       RPAREN                  )
10      PRINT                   print
10      LPAREN                  (
10      IDENTIFIER              p
10      DOT                     .
```

```
10      IDENTIFIER              x
10      RPAREN                  )
11      PRINT                   print
11      LPAREN                  (
11      IDENTIFIER              p
11      DOT                     .
11      IDENTIFIER              y
11      RPAREN                  )
14      IDENTIFIER              p
14      DOT                     .
14      IDENTIFIER              x
14      ASSIGN                  =
14      INT_LITERAL             10
15      IDENTIFIER              p
15      DOT                     .
15      IDENTIFIER              y
15      ASSIGN                  =
15      INT_LITERAL             20
16      PRINT                   print
16      LPAREN                  (
16      IDENTIFIER              p
16      DOT                     .
16      IDENTIFIER              x
16      RPAREN                  )
17      PRINT                   print
17      LPAREN                  (
17      IDENTIFIER              p
17      DOT                     .
17      IDENTIFIER              y
17      RPAREN                  )
20      LET                     let
20      IDENTIFIER              origin
20      ASSIGN                  =
20      IDENTIFIER              Point
20      LPAREN                  (
20      IDENTIFIER              x
20      COLON                   :
20      INT_LITERAL             0
20      COMMA                   ,
20      IDENTIFIER              y
20      COLON                   :
20      INT_LITERAL             0
20      RPAREN                  )
21      PRINT                   print
21      LPAREN                  (
21      IDENTIFIER              origin
21      DOT                     .
21      IDENTIFIER              x
21      RPAREN                  )
22      PRINT                   print
22      LPAREN                  (
22      IDENTIFIER              origin
22      DOT                     .
22      IDENTIFIER              y
22      RPAREN                  )
================================================================================
```

================================ PARSING MESSAGES ================================

```
  [PARSER] Struct declaration: Point
  [PARSER] Variable declaration (var with inference): p
  [PARSER] Print statement: print(t1)
  [PARSER] Print statement: print(t2)
  [PARSER] Assignment to struct member: p.x
```

```
  [PARSER] Assignment to struct member: p.y
  [PARSER] Print statement: print(t3)
  [PARSER] Print statement: print(t4)
  [PARSER] Variable declaration (let with inference): origin
  [PARSER] Print statement: print(t6)
  [PARSER] Print statement: print(t7)
  [PARSER] Program parsing completed successfully.
================================================================================


================================== SYMBOL TABLE ==================================
Identifier     Kind        Type              Scope     Line
--------------------------------------------------------------------------------
origin         let (const) Point             0         21
p              var (mut)   Point             0         10

--- Structs ---
struct Point {
    var x: Int
    var y: Int
} [Line 9]
================================================================================


======================== THREE ADDRESS CODE (TAC) ========================
1:        t0.x = 3
2:        t0.y = 4
3:        p = t0
4:        t1 = p.x
5:        print t1
6:        t2 = p.y
7:        print t2
8:        p.x = 10
9:        p.y = 20
10:       t3 = p.x
11:       print t3
12:       t4 = p.y
13:       print t4
14:       t5.x = 0
15:       t5.y = 0
16:       origin = t5
17:       t6 = origin.x
18:       print t6
19:       t7 = origin.y
20:       print t7
================================================================================


>>> Compilation finished successfully with 0 errors.
STATUS: Compilation SUCCEEDED without errors.

--------------------------------------------------------------------------------
TEST: Test 8: Semantic Error Diagnostics (tests/test8_semantic_errors.swift)
--------------------------------------------------------------------------------
Source Code:
// Test 8: Semantic Error Diagnostics

// 1. Immutability Violation: Reassignment to let
let maxLimit = 100
maxLimit = 200

// 2. Type Mismatch in Assignment
var age: Int = 25
age = "Twenty Five"
```

```
// 3. Undeclared Identifier
print(undeclaredVariable)

// 4. Duplicate Declaration in same scope
var value = 10
var value = 20

// 5. Non-boolean condition in if statement
let count = 5
if count {
    print("Invalid condition")
}

// 6. Subscripting a non-array or invalid index type
let flag = true
var items = [1, 2, 3]
let invalidItem = items["first"]

// 7. Non-existent struct member and mutating let struct
struct Box {
    var width: Int
}
let b = Box(width: 50)
b.width = 100
print(b.height)

Compiler Execution Output:
================================================================================
           SWIFT SUBSET COMPILER (LEX & YACC / FLEX & BISON)
================================================================================
Compiling file: tests/test8_semantic_errors.swift


================================ TOKEN STREAM ================================
Line    Token Type              Lexeme
--------------------------------------------------------------------------------
4       LET                     let
4       IDENTIFIER              maxLimit
4       ASSIGN                  =
4       INT_LITERAL             100
5       IDENTIFIER              maxLimit
5       ASSIGN                  =
5       INT_LITERAL             200
8       VAR                     var
8       IDENTIFIER              age
8       COLON                   :
8       TYPE_INT                Int
8       ASSIGN                  =
8       INT_LITERAL             25
9       IDENTIFIER              age
9       ASSIGN                  =
9       STRING_LITERAL          "Twenty Five"
12      PRINT                   print
12      LPAREN                  (
12      IDENTIFIER              undeclaredVariable
12      RPAREN                  )
15      VAR                     var
15      IDENTIFIER              value
15      ASSIGN                  =
15      INT_LITERAL             10
16      VAR                     var
16      IDENTIFIER              value
16      ASSIGN                  =
```

```
16      INT_LITERAL             20
19      LET                     let
19      IDENTIFIER              count
19      ASSIGN                  =
19      INT_LITERAL             5
20      IF                      if
20      IDENTIFIER              count
20      LBRACE                  {
21      PRINT                   print
21      LPAREN                  (
21      STRING_LITERAL          "Invalid condition"
21      RPAREN                  )
22      RBRACE                  }
25      LET                     let
25      IDENTIFIER              flag
25      ASSIGN                  =
25      BOOL_LITERAL            true
26      VAR                     var
26      IDENTIFIER              items
26      ASSIGN                  =
26      LBRACKET                [
26      INT_LITERAL             1
26      COMMA                   ,
26      INT_LITERAL             2
26      COMMA                   ,
26      INT_LITERAL             3
26      RBRACKET                ]
27      LET                     let
27      IDENTIFIER              invalidItem
27      ASSIGN                  =
27      IDENTIFIER              items
27      LBRACKET                [
27      STRING_LITERAL          "first"
27      RBRACKET                ]
30      STRUCT                  struct
30      IDENTIFIER              Box
30      LBRACE                  {
31      VAR                     var
31      IDENTIFIER              width
31      COLON                   :
31      TYPE_INT                Int
32      RBRACE                  }
33      LET                     let
33      IDENTIFIER              b
33      ASSIGN                  =
33      IDENTIFIER              Box
33      LPAREN                  (
33      IDENTIFIER              width
33      COLON                   :
33      INT_LITERAL             50
33      RPAREN                  )
34      IDENTIFIER              b
34      DOT                     .
34      IDENTIFIER              width
34      ASSIGN                  =
34      INT_LITERAL             100
35      PRINT                   print
35      LPAREN                  (
35      IDENTIFIER              b
35      DOT                     .
35      IDENTIFIER              height
35      RPAREN                  )
================================================================================
```

```
================================ PARSING MESSAGES ================================
  [PARSER] Variable declaration (let with inference): maxLimit
  [PARSER] Assignment to variable: maxLimit
  [PARSER] Variable declaration (var with type annotation): age: Int
  [PARSER] Assignment to variable: age
  [PARSER] Print statement: print(undeclaredVariable)
  [PARSER] Variable declaration (var with inference): value
  [PARSER] Variable declaration (var with inference): value
  [PARSER] Variable declaration (let with inference): count
  [PARSER] Print statement: print("Invalid condition")
  [PARSER] Variable declaration (let with inference): flag
  [PARSER] Variable declaration (var with inference): items
  [PARSER] Variable declaration (let with inference): invalidItem
  [PARSER] Struct declaration: Box
  [PARSER] Variable declaration (let with inference): b
  [PARSER] Assignment to struct member: b.width
  [PARSER] Print statement: print(t3)
  [PARSER] Program parsing completed successfully.
================================================================================


================================ SYMBOL TABLE ================================
Identifier     Kind        Type              Scope     Line
--------------------------------------------------------------------------------
age            var (mut)   Int               0         9
b              let (const) Box               0         34
count          let (const) Int               0         20
flag           let (const) Bool              0         26
invalidItem    let (const) Int               0         30
items          var (mut)   [Int]             0         27
maxLimit       let (const) Int               0         5
value          var (mut)   Int               0         16

--- Structs ---
struct Box {
    var width: Int
} [Line 33]
================================================================================


======================== THREE ADDRESS CODE (TAC) ========================
1:        maxLimit = 100
2:        maxLimit = 200
3:        age = 25
4:        age = "Twenty Five"
5:        print undeclaredVariable
6:        value = 10
7:        count = 5
8:        ifFalse count goto L0
9:        print "Invalid condition"
10:   L0:
11:       flag = true
12:       t0[0] = 1
13:       t0[1] = 2
14:       t0[2] = 3
15:       items = t0
16:       t1 = items["first"]
17:       invalidItem = t1
18:       t2.width = 50
19:       b = t2
20:       b.width = 100
21:       t3 = b.height
22:       print t3
```

```
================================================================================


============================== COMPILATION ERRORS ==============================
[Semantic Error at line 8]: Cannot assign to value: 'maxLimit' is a 'let' constant.
[Semantic Error at line 12]: Cannot assign value of type 'String' to type 'Int'.
[Semantic Error at line 12]: Cannot find identifier 'undeclaredVariable' in scope.
[Semantic Error at line 19]: Invalid redeclaration of identifier 'value' in current scope.
[Semantic Error at line 20]: Condition of 'if' statement must be 'Bool', got 'Int'.
[Semantic Error at line 27]: Array index must be of type 'Int', got 'String'.
[Semantic Error at line 35]: Cannot mutate member of immutable 'let' struct 'b'.
[Semantic Error at line 35]: Value of type 'Box' has no member 'height'.
Summary: 0 lexical, 0 syntax, 8 semantic errors.
================================================================================

STATUS: Compilation FAILED with errors.

--------------------------------------------------------------------------------
TEST: Test 9: Lexical & Syntactic Error Diagnostics (tests/test9_lexical_syntax_errors.swi
    ft)
--------------------------------------------------------------------------------
Source Code:
// Test 9: Lexical and Syntactic Error Diagnostics

// 1. Invalid Character (e.g. # or @ or $)
var #badIdentifier = 10

// 2. Unterminated String
let unclosed = "This string has no closing quote

// 3. Unterminated multi-line comment
/* This comment is never closed
var x = 5

Compiler Execution Output:
================================================================================
           SWIFT SUBSET COMPILER (LEX & YACC / FLEX & BISON)
================================================================================
Compiling file: tests/test9_lexical_syntax_errors.swift


================================ TOKEN STREAM ================================
Line    Token Type              Lexeme
--------------------------------------------------------------------------------
4       VAR                     var
4       IDENTIFIER              badIdentifier
4       ASSIGN                  =
4       INT_LITERAL             10
7       LET                     let
7       IDENTIFIER              unclosed
7       ASSIGN                  =
================================================================================


============================== PARSING MESSAGES ==============================
  [PARSER] Variable declaration (var with inference): badIdentifier
================================================================================


============================== SYMBOL TABLE ==============================
Identifier     Kind        Type              Scope     Line
--------------------------------------------------------------------------------
badIdentifier  var (mut)   Int               0         7
================================================================================
```

```
======================== THREE ADDRESS CODE (TAC) ========================
1:        badIdentifier = 10
===========================================================================


============================== COMPILATION ERRORS ==============================
[Lexical Error at line 4]: Unexpected character: ’#’ (ASCII: 35)
[Lexical Error at line 7]: Unterminated string literal: "This string has no closing quote
[Lexical Error at line 10]: Unterminated multi-line comment starting at line 10
[Syntax Error at line 12]: syntax error near ’’
Summary: 3 lexical, 1 syntax, 0 semantic errors.
================================================================================

STATUS: Compilation FAILED with errors.
```